



MINISTÈRIO DA EDUCAÇÃO UNIVERSIDADE FEDERAL DE ITAJUBÁ

Criada pela Lei nº 10.435 – 24/04/2002

Fundamentos de Programação

Manipulação de arquivos em C

Antes de começar a pensar na leitura de arquivos com a linguagem C, temos que entender alguns aspectos que estão relacionados com esta operação. A primeira coisa importante é entender qual a utilidade da leitura de um arquivo. Já vimos em outros exemplos que um programa pode receber dados externos vindo do usuário através da leitura do dado digitado no teclado. Para isto, podemos usar a função `scanf`, por exemplo. Mas em alguns casos, os dados necessários à solução de um problema estão armazenados em um arquivo.

Imagine que seu programa precisa ler informações (identidade, a idade e o sexo) de 3 pessoas. Se a entrada de dados fosse possível somente a partir do teclado, alguém deveria digitar esses dados. Esta é uma possível solução, mas não é a mais segura, uma vez que, ao digitar, a pessoa pode cometer uma variedade grande de erros. Além disso, há de se considerar o tempo gasto para isto. Portanto, seria interessante que sua aplicação (programa) fosse apta a ler diretamente esses dados de um arquivo.

Outra coisa importante é considerar a forma como os arquivos são manipulados no computador. Todo arquivo é armazenado na memória do computador. Sabemos que existem dois tipos de memória: permanente (HD, pendrive, disquete, etc ...) e volátil (RAM ou memória principal). A memória permanente funciona como uma memória de estoque, já a memória volátil é a memória de trabalho, ou seja, aquela usada pelo processador para executar as instruções.

Portanto, quando manipulamos um arquivo, é feita uma cópia do arquivo da memória permanente, como o HD, para memória principal. Um programa “trabalha” na cópia do arquivo na memória RAM. Depois de processadas as operações, a nova versão do arquivo com as alterações realizadas é enviada para memória permanente. Além disso, alguém pode abrir um arquivo sem necessariamente alterá-lo. De qualquer forma, quando lemos dados de um arquivo, é necessário que este arquivo seja “carregado” ou “aberto” na memória principal.

Agora vamos começar a falar especificamente da forma como faremos isso com a linguagem C. Sabemos que toda manipulação de dados realizada dentro de um programa é feita a partir da definição de variáveis e constantes. Logo se queremos manipular um arquivo, temos que definir variáveis que deverão assumir, em um determinado momento, o arquivo.

Uma variável em C precisa ser declarada. A declaração envolve a determinação do tipo. Por exemplo, quando declaramos `int idade`, o processador reserva um espaço da memória que será identificado logicamente pelo identificador `idade`. Neste caso, como `int` é um tipo primitivo, o processador sabe exatamente qual é o tamanho deste espaço a ser reservado.



MINISTÉRIO DA EDUCAÇÃO UNIVERSIDADE FEDERAL DE ITAJUBÁ

Criada pela Lei nº 10.435 – 24/04/2002

Em C, temos o tipo FILE que representa um arquivo, se fizéssemos **FILE arq**, a variável arq seria usada para manipular um arquivo (ou seja, o arquivo seria carregado no espaço de memória reservado para a variável arq).

Neste ponto, temos um impasse: qual tamanho da memória seria dedicado a esta variável?

Se fossemos obrigados a determinar o tamanho do arquivo ao declarar a variável, a utilização de arquivos seria muito limitada. Portanto, ao invés de declarar a variável arq como sendo um FILE, declaramos a variável arq como sendo um ponteiro para um arquivo. Em linhas gerais e abstraindo detalhes, arq aponta para o arquivo, ou seja, arq indica o endereço do arquivo na memória. Neste caso, arq deve ser declarada assim: FILE *arq, o asterisco diz que arq não é um arquivo, mas um ponteiro para um arquivo. O grande diferencial é que: como arq é um ponteiro para o arquivo, não precisamos determinar o espaço que o arquivo vai ocupar.

Uma coisa importante: quando declaramos uma variável do tipo ponteiro para um arquivo, ainda não temos um arquivo carregado na memória principal, ou seja, ainda não temos um arquivo aberto. Neste ponto, sabemos apenas que arq será a variável que vamos usar para manipular o arquivo na memória. Chamamos esta variável de identificador lógico do arquivo.

Mas este arquivo deve ser aberto, ou seja, o arquivo deve ser carregado da memória permanente para memória principal. Em C, temos uma função da biblioteca stdio.h que faz isso: o nome da função é fopen.

fopen precisa de duas informações (parâmetros de entrada): o nome do arquivo e o modo como este arquivo vai ser aberto. Um arquivo pode ser aberto somente para leitura (r), somente para escrita (w) ou para ambos (consulte a documentação da linguagem, para saber os diferentes modos de abertura de arquivo). No nosso caso, como vamos ler informação do arquivo, vamos abrir no modo leitura (r). Quando a função fopen não consegue abrir o arquivo, o retorno da função é NULL. Vamos a um exemplo onde o código será apresentado:

Suponha que tenhamos um arquivo chamado dados.dat contendo os seguintes dados:

```
3658912 35 F  
5489742 65 F  
4518721 28 M
```

Cada linha do arquivo diz respeito às informações de uma pessoa (chamamos isso de registro) e cada coluna representa, respectivamente, a identidade, a idade e o sexo das pessoas (chamamos isso de campo). Portanto, o arquivo dados.dat contém três registros, cada um contendo três campos.

Quando você criar seu programa na replit, você deve criar o arquivo na própria ferramenta. Para isso, selecione o botão new file, veja a Figura 1. Ao clicar nesse botão, você pode dar um nome para o arquivo e escrever os dados na tela de edição, como na Figura 2. Escreva um registro em cada linha.



MINISTÈRIO DA EDUCAÇÃO UNIVERSIDADE FEDERAL DE ITAJUBÁ

Criada pela Lei nº 10.435 – 24/04/2002

Depois de criar o arquivo, você deve selecionar a main para editar o código do seu programa, o arquivo ficará salvo.

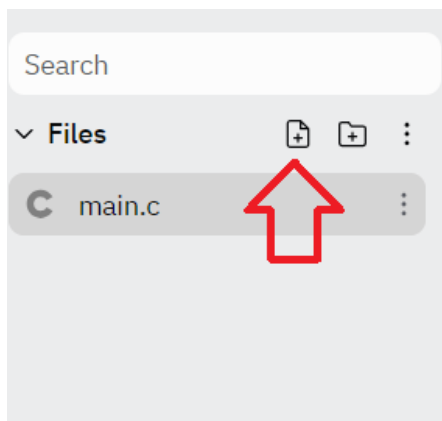


Figura 1 – Botão New File

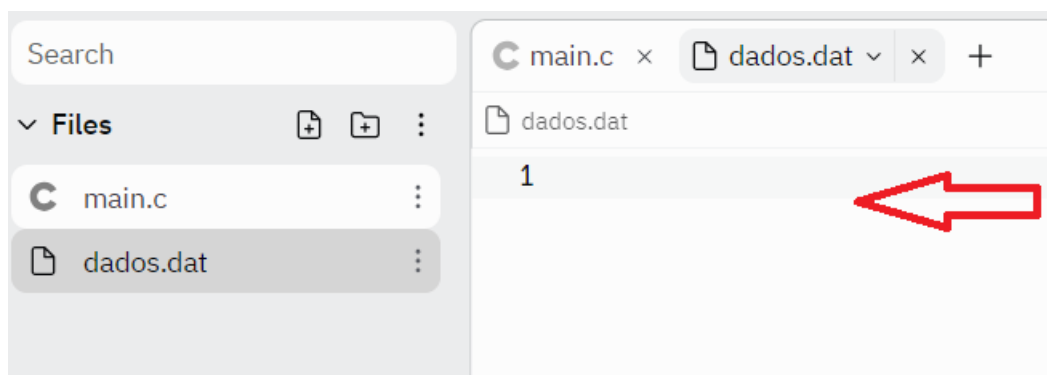


Figura 2 – Tela de Edição do Arquivo dados.dat que está selecionado

Voltando ao código do nosso programa, nossa primeira tarefa é ler os dados desse arquivo e escrever na tela as informações. Então declaramos uma variável que será um ponteiro para o arquivo na memória principal:

```
FILE *arq
```

Mas precisamos abrir o arquivo, faremos isso com a função `fopen` determinando o modo leitura.

```
arq = fopen("Dados.dat", r);
```

Como ilustrado na Figura 3, o arquivo chamado `dados.dat` será carregado na memória principal e identificado pelo ponteiro `arq`. Isso significa que `arq` contém um endereço da memória principal onde o arquivo está armazenado. No exemplo da figura, o valor de `arq` é `0x5072a0`, que é o endereço do arquivo.



MINISTÈRIO DA EDUCAÇÃO
UNIVERSIDADE FEDERAL DE ITAJUBÁ
Criada pela Lei nº 10.435 – 24/04/2002

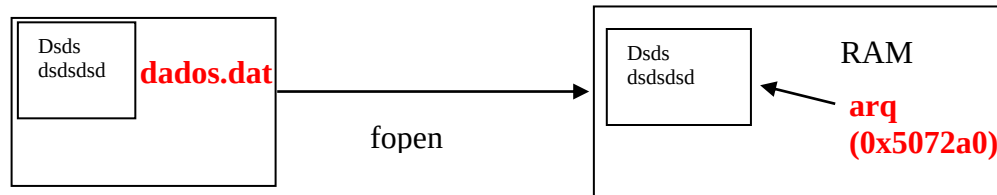


Figura 3 – Função fopen para abrir o arquivo na memória principal

Antes de usar o arquivo, temos que ter certeza de que tudo deu certo. Para isso, podemos avaliar o valor da variável `arq`, se `arq` for `NULL`, significa que a função `fopen` falhou ao tentar abrir o arquivo. Para isso, podemos fazer o seguinte teste:

```
arq = fopen("dado.dat","r");
if (arq == NULL){ // testa se foi possível abrir o arquivo
    printf("Não foi possível abrir o arquivo\n");
    return 1; // encerra o programa e retorna um código de erro = 1
}
```

Se `arq` não é `NULL`, nosso arquivo está aberto e podemos ler as informações. No nosso exemplo, para cada linha (registro), temos três informações diferentes: identidade, idade e o sexo. Ao ler o arquivo, estas informações deverão ser manipuladas isoladamente. Como vamos fazer isso? Vamos declarar variáveis que deverão receber o valor de cada campo toda vez que um registro for lido.

Então voltamos à declaração, agora teremos:

```
FILE *arq
int idade, identidade;
char sexo;
```

Neste ponto, precisamos ler o arquivo que já está aberto na memória. Agora vamos usar uma nova função: `fscanf`. As funções `fscanf` e `scanf` são semelhantes, `scanf` lê do teclado, `fscanf` lê do arquivo. A `fscanf` precisa dos seguintes argumentos:

- Nome lógico do arquivo (variável que aponta para o arquivo)
- Formato dos campos que serão lidos
- Endereço das variáveis que irão assumir os valores dos campos

No nosso caso, teremos:

```
fscanf(arq, "%d %d %c", &identidade, &idade, &sexo);
```



MINISTÉRIO DA EDUCAÇÃO UNIVERSIDADE FEDERAL DE ITAJUBÁ

Criada pela Lei nº 10.435 – 24/04/2002

Como temos três registros (linhas), vamos precisar executar esta ação três vezes. Nosso código fica assim:

```
#include <stdio.h>
int main(void) {
    FILE *arq;
    int id, idade;
    char tipo;
    arq = fopen("dado.dat", "r");
    if (arq == NULL) { // testa se foi possível abrir o arquivo
        printf("Não foi possível abrir o arquivo\n");
        return 1;
    }
    printf("Os dados do arquivo são:\n");
    for (int i = 0; i < 3; i++) {
        fscanf(arq, "%d %d %c", &id, &idade, &tipo);
        printf("%d %d %c\n", id, idade, tipo);
    }
    return 0; }
```

Vamos agora a um exemplo um pouco diferente, queremos saber qual é a pessoa mais velha. Para isso, vamos precisar ler o arquivo e, para cada registro lido, temos que verificar se a idade é a maior, se for, temos que “guardar” a idade e a identidade.

```
#include <stdio.h>
int main () {
    int idade, identidade, i, iMaior, idMaior;
    char sexo;
    FILE *arq;
    iMaior = -1;
    arq = fopen("dados.dat", "r");
    if (arq == NULL) { // testa se foi possível abrir o arquivo
        printf("Não foi possível abrir o arquivo\n");
        return 1;
    }
    for (i=0; i<3;i++) {
        fscanf(arq, "%d %d %c", &identidade, &idade, &sexo);
        if (idade > iMaior) {
            iMaior = idade;
            idMaior = identidade;
        }
    }
    printf("\n A pessoa mais velha tem %d anos e a identidade é: %d\n", iMaior, idMaior); }
```



MINISTÈRIO DA EDUCAÇÃO
UNIVERSIDADE FEDERAL DE ITAJUBÁ

Criada pela Lei nº 10.435 – 24/04/2002

Nos exemplos anteriores, estamos supondo que a quantidade de linhas do arquivo é conhecida (3 linhas). Mas essa suposição é difícil acontecer na prática. Por isso, podemos usar a função `feof()`, essa função recebe um ponteiro para um arquivo como parâmetro e retorna 0 (falso) quando ainda não é o fim do arquivo. Vale lembrar que em C, o valor 0 é sempre falso em um teste lógico, logo, no código abaixo, caso não seja o fim do arquivo, a função retorna 0 (falso), `!(0) = !(falso) = verdadeiro`. Você pode ler assim: “enquanto não é o fim do arquivo ...”. Nosso código fica assim:

```
#include <stdio.h>
int main () {
    int idade, identidade, i, iMaior, idMaior;
    char sexo;
    FILE *arq;
    iMaior = -1;
    arq = fopen("Dados.txt","r");
    if (arq == NULL){ // testa se foi possível abrir o arquivo
        printf("Não foi possível abrir o arquivo\n");
        return 1;
    }
    fscanf(arq, "%d %d %c", &identidade, &idade, &sexo);
    while (!feof(arq)) {
        if (idade > iMaior) {
            iMaior = idade;
            idMaior = identidade;
        }
        fscanf(arq, "%d %d %c", &identidade, &idade, &sexo);
    }
    printf("\n A pessoa mais velha tem %d %d\n", iMaior, idMaior);
    fclose(arq); }
```

Sabemos ler informações do arquivo, agora precisamos escrever informações no arquivo. No exemplo seguinte, vamos criar um arquivo com informações de 3 produtos contendo os seguintes dados: código do produto (inteiro) e nome do produto (máximo 50 caracter). Como vamos escrever no arquivo, precisamos usar um modo de abertura diferente dos exemplos anteriores. O modo de abertura `w`, da função `fopen`, cria um arquivo para escrita. Porém, se o arquivo já existir, a função apaga o arquivo e cria novamente, o que significa que o conteúdo do arquivo, caso ele já exista, será apagado. O modo de abertura `w+`, cria um arquivo para escrita e leitura.

Além de abrir o arquivo para escrita e leitura, vamos usar a função `fprintf` que, assim como a função `printf`, é usada para escrever uma informação, a diferença é que a escrita é feita em um arquivo apontado por `arq` (ponteiro passado como parâmetro).



MINISTÉRIO DA EDUCAÇÃO UNIVERSIDADE FEDERAL DE ITAJUBÁ

Criada pela Lei nº 10.435 – 24/04/2002

Conhecemos outra nova função no exemplo, a função `rewind`. Depois de escrever a informação no arquivo com a `fprintf`, estaremos no fim do arquivo. Como queremos fazer a leitura desde o início do arquivo, temos que reposicionar o ponteiro. Vamos fazer isso com a função `rewind`. Depois de voltar para o começo do arquivo, processamos a leitura como foi feito no primeiro exemplo: lemos o conteúdo do arquivo, armazenamos os valores de cada linha nas variáveis declaradas (código e nome) e imprimimos os valores dessas variáveis.

```
#include<stdio.h>
int main () {
    int i, codigo;
    char nome[50];
    FILE *arq;
    arq = fopen("produtos.dat","w+"); //abre o arquivo para leitura e escrita
    if (arq == NULL) { //se não foi possível abrir o arquivo, arq será NULL
        printf("\nArquivo inválido");
        return 1; // interrompe o programa
    }
    for (i=0;i<3;i++){
        printf("\bDigite um numero: ");
        scanf("%d", &codigo);
        printf("\bDigite o nome: ");
        scanf(" %[^\n]", nome);
        fprintf(arq,"%d %s \n",codigo,nome);
    }
    rewind(arq); //reposiciona o ponteiro no começo do arquivo
    fscanf(arq, "%d %[^\n]", &codigo,nome);
    while(!feof(arq)) {
        printf("%d %s\n", codigo,nome);
        fscanf(arq, "%d %[^\n]", &codigo,nome);
    }
}
```

Na replit, depois de executar seu programa, você vai perceber que o arquivo `produtos.dat` foi criado. Você pode acessar o conteúdo do arquivo selecionando o nome do arquivo como na Figura 4.



MINISTÈRIO DA EDUCAÇÃO UNIVERSIDADE FEDERAL DE ITAJUBÁ

Criada pela Lei nº 10.435 – 24/04/2002

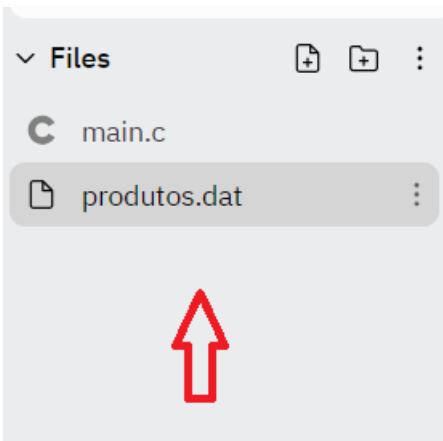


Figura 4 – Arquivo criado com fopen no modo w+

Agora vamos pensar mais um pouco, imagine que os dados que vamos manipular sejam de um conjunto de 3 clientes. Cada cliente é definido pelo seu código e seu nome. Portanto, temos uma informação estruturada nesse problema e, por isso, vamos definir o cliente como um struct. Para manipular o struct em um arquivo, podemos usar as funções fwrite e fread. Essas funções permitem escrever blocos de dados.

A função fwrite escreve um dado no arquivo e a função fread vai ler dados de um arquivo, as duas precisam de quatro parâmetros:

- endereço do dado que vai ser gravado ou lido (no nosso caso, é o endereço da estrutura que vai ser gravada ou lida do arquivo)
- tamanho do dado que vai ser gravado ou lido, a função sizeof (var) retorna o tamanho da variável (var) que é passada como parâmetro (no nosso caso é tamanho da struct definida)
- total de unidades de dados que serão gravadas ou lidas (no nosso caso, vamos manipular 1 única estrutura por vez)
- ponteiro para o arquivo

Nosso código fica assim:



MINISTÈRIO DA EDUCAÇÃO
UNIVERSIDADE FEDERAL DE ITAJUBÁ

Criada pela Lei nº 10.435 – 24/04/2002

```
#include<stdio.h>
typedef struct cliente {
    int codigo;
    char nome[50];
} tcliente;
int main(){
    int i;
    tcliente c;
    FILE *arq;
    arq = fopen("clientes.dat","w+");
    if (!arq){
        printf("\nErro ao abrir o arquivo");
        return 1;
    }
    for (i=0;i<3;i++){
        printf("\nDigite o nome:");
        scanf(" %[^\\n]", c.nome);
        printf("\nDigite a codigo:");
        scanf(" %d", &c.codigo);
        fwrite(&c, sizeof(c), 1, arq); //vamos escrever um cliente no arquivo
    }
    rewind(arq);
    fread(&c, sizeof(c), 1, arq); // vamos ler 1 cliente do arquivo
    while(!feof(arq)) {
        printf("\nCodigo %d - Nome: %s",c.codigo, c.nome);
        fread(&c, sizeof(c), 1, arq);
    }
    fclose(arq);}
```

É importante observar que o arquivo criado com a função `fwrite` não é um arquivo texto, essa função cria um arquivo binário. Em um arquivo criado no modo binário, não existe a noção de linha, os arquivos são lidos e escritos em blocos.

Agora vamos analisar o código seguinte. Vejam que a mesma informação foi criada usando estratégias diferentes.

- Estratégia 1: escrever os dados no arquivo texto com `fprintf` e ler com `fscanf`
- Estratégia 2: escrever os dados no arquivo binário com `fwrite` e ler com `fread`



MINISTÉRIO DA EDUCAÇÃO
UNIVERSIDADE FEDERAL DE ITAJUBÁ

Criada pela Lei nº 10.435 – 24/04/2002

Depois de executar o código, compare o conteúdo dos dois arquivos na replit.

```
#include<stdio.h>
typedef struct cliente {
    int codigo;
    char nome[50];
} tcliente;
int main(){
    int i;
    tcliente c;
    FILE *arq, *arq1;
    arq = fopen("clientes.dat","w+");
    arq1 = fopen("clientes2.dat","w+");
    if (!arq || !arq1){
        printf("\nErro ao abrir o arquivo");
        return 1;
    }
    for (i=0;i<3;i++){
        printf("\nDigite o nome:");
        scanf(" %[^\\n]", c.nome);
        printf("\nDigite a codigo:");
        scanf(" %d", &c.codigo);
        fwrite(&c, sizeof(c), 1, arq); //vamos escrever um cliente no arquivo
        fprintf(arq1,"%d %s\\n", c.codigo,c.nome);
    }
    rewind(arq); rewind(arq1);
    fread(&c, sizeof(c), 1, arq);
    while(!feof(arq)) {
        printf("\nCodigo %d - Nome: %s",c.codigo, c.nome);
        fread(&c, sizeof(c), 1, arq);
    }
    fscanf(arq1,"%d %s",&c.codigo,c.nome);
    while(!feof(arq1)) {
        printf("\nCodigo %d - Nome: %s",c.codigo, c.nome);
        fscanf(arq1,"%d %s",&c.codigo,c.nome);
    }
    fclose(arq);
    fclose(arq1);
}
```

Para finalizar, falta apenas falar da função fclose, essa função fecha o arquivo que foi aberto com a fopen. Depois de executar a função fclose, o espaço de memória ocupado pelo arquivo é liberado.